

CS 551 Systems Programming

Lecture 7: Structs and Methods

Where we are

You have enums, which say "**one of these.**" Structs say "**all of these at once.**"

Between them, those two constructs are how you build every data type you will ever write in Rust. There are structs, enums, and traits, with traits next lecture.

One thing that is genuinely absent: **no struct inherits data from another struct.** There is no `struct Dog: Animal`; it does not parse. If a `Dog` needs a `Name`, it holds one as a field.

Be careful with the slogan "Rust has no inheritance," which you will see repeated online. Traits *do* build on other traits, and we will do exactly that next lecture. What Rust lacks is inheritance of **data**, not of behavior.

Today: defining your own types, deciding what they own, and giving them behavior.

Today's path

1. **Defining data.** Structs, and the three flavors of them.
2. **Ownership in fields.** Why your struct fields are `String` and not `&str`.
3. **Behavior.** `impl` blocks, methods, and associated functions.
4. `derive`. Free implementations, and a first look at traits.

1. Defining data

A struct is named fields with types

```
1 struct User {  
2     username: String,  
3     email: String,  
4     sign_in_count: u64,  
5     active: bool,  
6 }
```

That is a type definition, not a value. It says: a `User` is exactly these four things, with these four types, and every `User` has all of them.

Compare to C, where this is `struct User { char *username; ... }` and it is on you to remember who frees the strings. The types here say who owns what. We come back to that in part 2.

Creating one

```
1 fn main() {
2     let user1 = User {
3         username: String::from("jeremy"),
4         email: String::from("jblackbu@binghamton.edu"),
5         sign_in_count: 1,
6         active: true,
7     };
8
9     println!("{}", user1.username, user1.email);
10 }
```

Every field must be given a value. There is no partially-initialized struct, no default zeroing, and no way to read a field that was never set. If you want defaults, you ask for them explicitly. We will see how.

Field order in the literal does not have to match the definition.

Mutability is per binding, not per field

```
1 fn main() {
2     let mut user1 = User {
3         username: String::from("jeremy"),
4         email: String::from("jblackbu@binghamton.edu"),
5         sign_in_count: 1,
6         active: true,
7     };
8
9     user1.sign_in_count += 1;    // fine: the binding is mut
10    println!("{}", user1.sign_in_count);
11 }
```

There is no `mut` on individual fields. Either the whole binding is mutable or none of it is.

This surprises people who expect C++'s per-member `const`. Rust's answer is that mutability belongs to whoever holds the value, not to the type.

Field init shorthand

When a variable already has the field's name, say it once:

```
1 fn build_user(username: String, email: String) -> User {
2     User {
3         username,           // instead of username: username
4         email,              // instead of email: email
5         sign_in_count: 1,
6         active: true,
7     }
8 }
9
10 fn main() {
11     let u = build_user(String::from("jeremy"), String::from("jblackbu@binghamton.edu"));
12     println!("{}", u.username);
13 }
```

Struct update syntax

Build a new value from an old one:

```
1 fn main() {
2     let user1 = User {
3         username: String::from("jeremy"),
4         email: String::from("jblackbu@binghamton.edu"),
5         sign_in_count: 1,
6         active: true,
7     };
8     let user2 = User {
9         email: String::from("other@example.edu"),
10        ..user1           // take the rest from user1
11    };
12    println!("{}", user2.username, user2.sign_in_count);
13 }
```

`..user1` fills in every field you did not name.

Careful: `..` moves

```
1 fn main() {
2     let user1 = User {
3         username: String::from("jeremy"),
4         email: String::from("jblackbu@binghamton.edu"),
5         sign_in_count: 1,
6         active: true,
7     };
8     let user2 = User { email: String::from("other@example.edu"), ..user1 };
9     println!("{}", user1.username);    // user1.username moved into user2
10 }
```

```
error[E0382]: borrow of moved value: `user1.username`
```

`username` is a `String`, so `..user1` **moved** it. `sign_in_count` and `active` are `Copy`, so those were copied and are still readable.

Tuple structs

Fields without names, when names would not add anything:

```
1 struct Point(i32, i32);
2 struct Meters(f64);
3
4 fn main() {
5     let origin = Point(0, 0);
6     let dist = Meters(9.8);
7
8     println!("{}", origin.0, origin.1, dist.0);
9 }
```

`Point` and `Meters` are **distinct types** even though `Meters` is just an `f64` underneath. A function taking `Meters` will not accept a bare `f64`, or a `Feet`.

The newtype pattern

```
1 struct Point(i32, i32);
2 struct Meters(f64);
3
4 fn main() {
5     let origin = Point(0, 0);
6     let dist = Meters(9.8);
7
8     println!("{}", origin.0, origin.1, dist.0);
9 }
```

This single-field form is called the **newtype pattern**, and it is one of the most useful things in the language. We come back to it at the end.

Unit structs

A struct with no fields at all:

```
1 struct AlwaysEqual;
2
3 fn main() {
4     let _subject = AlwaysEqual;
5 }
```

This looks pointless and mostly is, until next lecture. A type with no data can still carry **behavior**, because you can implement traits on it. That is how marker types and zero-sized strategy objects work, and they cost zero bytes at runtime.

2. Ownership in fields

Why `String` and not `&str`

The deliberate choice

Look again at `User`:

```
1 struct User {  
2     username: String,      // not &str  
3     email: String,        // not &str  
4     ...  
5 }
```

Using `String` means each `User` **owns** its text. The struct holds a heap allocation, and when the `User` is dropped, the strings go with it.

That is a decision, not an accident. It guarantees a `User` is always valid on its own, never pointing at something that might already be gone.

The alternative sounds cheaper. Let's try it.

LIVE CODING CHALLENGE

Write the `User` struct with `username` and `email` as `&str` instead of `String`.

What does the compiler ask you for?

Pause here. Try it before the next slide.

What happens with `&str`

```
1 struct User {  
2     username: &str,  
3     email: &str,  
4     sign_in_count: u64,  
5     active: bool,  
6 }
```

```
error[E0106]: missing lifetime specifier  
|  
|     username: &str,  
|               ^ expected named lifetime parameter  
help: consider introducing a named lifetime parameter  
|  
| struct User<'a> {  
|     username: &'a str,
```

The compiler is asking a question you cannot yet answer.

How long does the borrowed text live?

Relative to the struct, that is. A struct holding a reference must promise it **does not outlive what it points at**.

Saying that requires a lifetime parameter, which is Lecture 5's last slide and not something you want on your first struct. `String` sidesteps the question by owning the text.

Welcome to lifetimes

You *can* write it. `struct User<'a> { username: &'a str }` compiles, and it means "a `User` that may not outlive the strings it borrows."

We are not doing that today, for a reason that is worth stating: **it is rarely what you want in an application type**. A borrowing struct infects everything that holds it with the same lifetime constraint: you end up threading `'a` through half your program to save one allocation.

The rule of thumb, which holds for the projects:

- **Owning fields (`String`, `Vec<T>`) by default.** Simple, always valid.
- **Borrowing fields (`&'a str`, `&'a [T]`) when the struct is a short-lived view** over data someone else owns, like a parser working over a buffer.

We cover lifetimes properly later. Today, own your fields.

3. Behavior

`impl`, methods, and associated functions

derive gives you implementations for free

```
1 #[derive(Debug)]
2 struct Rectangle {
3     width: u32,
4     height: u32,
5 }
6 fn main() {
7     let rect = Rectangle { width: 30, height: 50 };
8     println!("{rect:?}");      // Rectangle { width: 30, height: 50 }
9     println!("{rect:#?}");     // the same, pretty-printed over several lines
10 }
```

derive gives you implementations for free

```
1 #[derive(Debug)]
2 struct Rectangle
3     width: u32,
4     height: u32,
5 }
6 fn main() {
7     let rect = Rectangle { width: 30, height: 50 };
8     println!("{rect:?}");           // Rectangle { width: 30, height: 50 }
9     println!("{rect:#?}");         // the same, pretty-printed over several lines
10 }
```

An **attribute**. It asks the compiler to write an implementation for you

Without it that `println!` does not compile: no `Rectangle` knows how to print itself.

`derive` writes an implementation for you. `{:?}",` is the debug format, `{:#?}"` is the multi-line version. Use them constantly while developing.

What `derive` actually is

`#[derive(Debug)]` implements the **Debug trait** for your type. You can only derive when every field also has the trait, so a struct holding a `String` cannot be `Copy`:

Derive	Gives you
<code>Debug</code>	<code>{:?}</code> formatting
<code>Clone</code>	<code>.clone()</code> , an explicit deep copy
<code>Copy</code>	implicit copying instead of moving (needs <code>Clone</code>)
<code>PartialEq</code> , <code>Eq</code>	<code>==</code> and <code>!=</code>
<code>PartialOrd</code> , <code>Ord</code>	<code><</code> , <code>></code> , and <code>.cmp()</code>
<code>Default</code>	<code>Type::default()</code>
<code>Hash</code>	usable as a <code>HashMap</code> key

Methods live in an `impl` block

```
1  impl Rectangle {
2      fn area(&self) -> u32 {
3          self.width * self.height
4      }
5
6      fn can_hold(&self, other: &Rectangle) -> bool {
7          self.width >= other.width && self.height >= other.height
8      }
9  }
10
11 fn main() {
12     let a = Rectangle { width: 30, height: 50 };
13     let b = Rectangle { width: 10, height: 40 };
14     println!("{}", a.area(), a.can_hold(&b));
15 }
```

Methods live in an `impl` block

```
1  impl Rectangle {
2      fn area(&self) {
3          self.width * self.height
4      }
5
6      fn can_hold(&self, other: &Rectangle) -> bool {
7          self.width >= other.width && self.height >= other.height
8      }
9  }
10
11 fn main() {
12     let a = Rectangle { width: 30, height: 50 };
13     let b = Rectangle { width: 10, height: 40 };
14     println!("{}", a.area(), a.can_hold(&b));
15 }
```

To say a function can be called on instances of a struct, you put it in an `impl` block

The first parameter is `&self`. That is what makes it a method rather than a plain function

The three forms of `self`

```
1 fn area(&self) -> u32           // borrows: read the value
2 fn resize(&mut self, w: u32)    // borrows mutably: change it
3 fn consume(self) -> Vec<u32>    // takes ownership: the value is gone after
```

These are exactly the three things you already know from Lecture 5, applied to the receiver. `&self` is shorthand for `self: &Self`.

Why `&self` and not `self`? Because `self` would *move* the struct into the method, and the caller could not use it afterwards. Almost every method you write takes `&self`. Take `&mut self` when you mutate, and bare `self` only when the method genuinely consumes the value, like a builder's `.build()`.

LIVE CODING CHALLENGE

Give `Rectangle` a `double` method that doubles both fields in place.

What is its `self` parameter, and what does that force on the caller?

Pause here. Try it before the next slide.

Mutating through `&mut self`

```
1  impl Rectangle {
2      fn double(&mut self) {
3          self.width *= 2;
4          self.height *= 2;
5      }
6  }
7
8  fn main() {
9      let mut rect = Rectangle { width: 3, height: 4 };
10     rect.double();
11     println!("{rect:?}");    // Rectangle { width: 6, height: 8 }
12 }
```

Note `let mut rect`. Calling a `&mut self` method requires a mutable binding, for exactly the reason Lecture 5 gave: you cannot hand out an exclusive borrow of something you only have shared access to.

Associated functions have no `self`

```
1  impl Rectangle {
2      fn new(width: u32, height: u32) -> Self {
3          Rectangle { width, height }
4      }
5
6      fn square(size: u32) -> Self {
7          Rectangle { width: size, height: size }
8      }
9  }
10
11 fn main() {
12     let r = Rectangle::new(3, 4);
13     let s = Rectangle::square(5);
14     println!("{r:?} {s:?}");
15 }
```

Associated functions have no `self`

```
1  impl Rectangle {  
2      fn new(width: u32, height: u32) -> Self {  
3          Rectangle { width, height }  
4      }  
5  
6      fn square(size: u32) -> Self {  
7          Rectangle { width: size, height: size }  
8      }  
9  }  
10  
11 fn main() {  
12     let r = Rectangle::new(3, 4);  
13     let s = Rectangle::square(5);  
14     println!("{r:?} {s:?}");  
15 }
```

A function associated with the struct, but not with any particular instance of it. No `self` parameter

Which is why you call it through the type, with `::`

Associated functions belong to a **type**

No `self` parameter means it belongs to the **type**, not to an instance. Call it with `::` rather than `.`.

You have used these since Lecture 1: `String::new`, `String::from`, `Vec::new`. There is no `new` keyword in Rust; `new` is just a conventional name for a constructor.

Self is a shorthand

Inside an `impl` block, `Self` means "the type this block is for."

```
1 impl Rectangle {  
2     fn new(width: u32, height: u32) -> Self {      // Self == Rectangle  
3         Self { width, height }  
4     }  
5 }
```

Prefer `Self` in return types and constructors. If you rename the struct, the `impl` block does not change.

You can have several `impl` blocks

```
1  impl Rectangle {
2      fn area(&self) -> u32 {
3          self.width * self.height
4      }
5  }
6
7  impl Rectangle {
8      fn perimeter(&self) -> u32 {
9          2 * (self.width + self.height)
10     }
11 }
12
13 fn main() {
14     let r = Rectangle { width: 3, height: 4 };
15     println!("{}", r.area(), r.perimeter());
16 }
```

Who cares?

There is no requirement to put everything in one block. This matters more than it might first seem: next lecture you will write `impl SomeTrait for Rectangle`, which is *necessarily* a separate block.

4. The newtype pattern

A struct that wraps exactly one thing

Wrapping a value to give it meaning

```
1 struct Meters(f64);
2 struct Feet(f64);
3
4 fn describe(distance: Meters) -> String {
5     format!("{}", distance.0)
6 }
7
8 fn main() {
9     println!("{}", describe(Feet(9.8)));
10 }
```

Wrapping a value to give it meaning

```
1 struct Meters(f64);
2 struct Feet(f64);
3
4 fn describe(distance: Meters) -> String {
5     format!("{}", distance.0)
6 }
7
8 fn main() {
9     println!("{}", describe(Feet(9.8)));
10 }
```



Will it compile?

What's the point?

```
error[E0308]: mismatched types
  |           println!("{}", describe(Feet(9.8)));
  |                                   ^^^^^^^^^^ expected `Meters`, found `Feet`
```

Both are an `f64` at runtime, and the wrapper costs **nothing**: same size, same layout, no indirection. The distinction exists only at compile time, which is exactly where you want it.

Why you will actually use this

Three reasons, all of which show up in the projects:

1. **Meaning.** `Meters` and `Feet` are not interchangeable. Neither are `PasswordLength` and `ThreadCount`, even though both are numbers.
2. **Invariants.** If the only way to build a `NonEmptyPassword` is through a constructor that checks, then holding one is proof the check happened. The type carries the guarantee.
3. **Implementing traits on foreign types.** You cannot implement your trait on `Vec<T>` if you own neither. Wrap it in a newtype and you can. This is called the orphan rule, and it is next lecture.

Recap

1. **Structs are named fields.** All fields, always initialized, no partial values.
2. **Mutability is per binding**, not per field.
3. `..other` **fills in the rest**, and it *moves* non-`Copy` fields.
4. **Own your fields.** `String`, not `&str`, unless the struct is a short-lived view. A borrowing struct needs a lifetime parameter, and it spreads.
5. `derive` **writes implementations for you.** `Debug` first, and `{:?}` / `{:#?}` constantly.
6. `impl` **blocks hold methods.** `&self` to read, `&mut self` to change, `self` to consume.
7. **No `self` means an associated function**, called with `::`. `new` is a convention, not a keyword.
8. **Newtypes cost nothing** and buy you type safety, invariants, and trait impls.

Exercises

1. Give `Rectangle` an `is_square(&self) -> bool`. Then add `Rectangle::unit()` returning a `1x1`. Which one is a method and which is an associated function, and why?
2. Add `#[derive(Clone, PartialEq)]` to `Rectangle` and compare two with `==`. Now try adding `Copy`. Then change `width` to a `String` and try `Copy` again. Explain the error.
3. Write `fn consume(self) -> u32` on `Rectangle` returning the area. Call it twice on the same value. Read the error, and say which of Lecture 5's rules it is.
4. Take the `User` struct and change `username` to `&str`. Follow the compiler's suggestion all the way until it compiles. How many places did you have to touch?
5. Write a newtype `Celsius(f64)` and a `to_fahrenheit` method. Then try to pass a bare `f64` to it.
6. Predict, then check: does `let r = Rectangle { width: 3, height: 4 }; r.double();` compile?

Reading

- **The Rust Programming Language**, chapter 5 (Using Structs to Structure Related Data), all three sections.
- **Rust by Example**, the sections on structs and on `impl`.
- **The Rust API Guidelines**, on constructors and naming. Short, and it will make your project code look like everyone else's, which is the goal.

Next lecture: traits. What `derive` was doing all along, and the only mechanism Rust has for shared behavior.

Next: Lecture 8

Traits

Shared behavior, and what `derive` was doing all along.